Aprendizaje de Máquina
Sesión 17: Aprendizaje de máquina para textos

Angel Rodríguez

Sección 4. Aprendizaje de máquina para textos.

Utilizaremos principalmente el libro de

Jurafsky y Martin, *Speech and Language Processing*.

Tema 4.1: Introducción al procesamiento de textos

Hasta ahora hemos trabajado principalmente con datos que ya tienen una **representación numérica**:

$$x = (x_1, x_2, \ldots, x_p).$$

Sin embargo, un algoritmo de aprendizaje de máquina no puede utilizar directamente un documento como

"La película estuvo excelente".

Necesitamos construir un flujo de procesamiento:

texto crudo $\longrightarrow$ preprocesamiento $\longrightarrow$ tokens $\longrightarrow$ representación numérica

En este tema estudiaremos principalmente las primeras etapas del proceso.

¿Qué es una palabra?

A primera vista, podríamos separar un texto utilizando los espacios.

```
El aprendizaje de máquina es interesante
```

Pero aparecen inmediatamente algunas decisiones:

- ¿La puntuación constituye una unidad independiente?
- ¿Mayúsculas y minúsculas representan la misma palabra?
- ¿Cómo tratamos contracciones o palabras compuestas?
- ¿Qué ocurre en lenguas donde los espacios no marcan las fronteras entre palabras?

Idea importante

La definición de “palabra” depende de la tarea y del lenguaje. Por ello necesitamos definir explícitamente las unidades que procesará el modelo.

Vocabulario: tipos e instancias

Consideremos:

"el gato mira al gato".

Podemos distinguir:

- **Instancias de palabras**: todas las apariciones en el texto.
- **Tipos de palabras**: las palabras distintas que aparecen.

En el ejemplo:

$$N = 5, \qquad V = \{\text{el, gato, mira, al}\}, \qquad |V| = 4.$$

Vocabulario

El conjunto de unidades distintas disponibles para representar el texto se denomina **vocabulario**. Su tamaño se denota normalmente por $|V|$.

Esta idea será fundamental cuando transformemos documentos en vectores.

El problema de las palabras desconocidas

Supongamos que construimos el vocabulario utilizando los datos de entrenamiento:

$$V = \{\texttt{low}, \texttt{new}, \texttt{newer}, \ldots\}.$$

Después aparece en un nuevo documento:

$$\texttt{lower}.$$

Si utilizamos exclusivamente palabras completas, `lower` no pertenece al vocabulario.

$$\texttt{lower} \notin V.$$

Problema

No importa qué tan grande sea el corpus (es decir la *colección de textos*) de entrenamiento: aparecerán palabras que el modelo no observó durante el entrenamiento.

Una solución moderna consiste en trabajar también con unidades **más pequeñas que una palabra**: las **subpalabras**.

Tokenización

La **tokenización** transforma una cadena de texto en una secuencia de unidades que el modelo puede procesar:

$$\text{texto} \longrightarrow [t_1, t_2, \ldots, t_n].$$

Los tokens pueden corresponder a:

- palabras completas,
- partes frecuentes de palabras (**subwords**),
- y, en casos extremos, caracteres individuales.

Por ejemplo, una palabra no observada puede representarse mediante unidades conocidas:

$$\texttt{lower} \longrightarrow [\texttt{low}, \texttt{er}].$$

Ventaja

Un vocabulario de subpalabras permite representar palabras nuevas combinando tokens que el modelo ya conoce.

Byte-Pair Encoding (BPE)

Un algoritmo ampliamente utilizado para construir un vocabulario de subpalabras es **Byte-Pair Encoding (BPE)**.

BPE puede entenderse en dos etapas:

1. **Entrenamiento:** a partir de un corpus, se aprende un vocabulario de tokens.
2. **Codificación:** un nuevo texto se divide utilizando el vocabulario aprendido.

$$\text{corpus de entrenamiento} \xrightarrow{\text{BPE trainer}} V$$

$$\text{nuevo texto} + V \xrightarrow{\text{BPE encoder}} [t_1, t_2, \ldots, t_n]$$

La idea del entrenamiento es muy simple: **combinar repetidamente pares de tokens frecuentes**.

BPE: ejemplo de entrenamiento

Supongamos que el corpus contiene inicialmente los caracteres:

`A B D C A B E C A B`

con vocabulario

$$V = \{A, B, C, D, E\}.$$

El par adyacente más frecuente es `A B`. Lo combinamos:

`AB D C AB E C AB`

$$V = \{A, B, C, D, E, AB\}.$$

Ahora el par más frecuente es `C AB`:

`AB D CAB E CAB`

$$V = \{A, B, C, D, E, AB, CAB\}.$$

El procedimiento continúa hasta realizar k fusiones, es decir hasta generar k tokens nuevos.

BPE: algoritmo conceptual

1. Inicializar el vocabulario con unidades pequeñas, por ejemplo caracteres.
2. Contar pares de tokens adyacentes en el corpus.
3. Encontrar el par más frecuente.
4. Fusionar ese par y agregar el nuevo token al vocabulario.
5. Repetir hasta alcanzar el número deseado de fusiones.

Así obtenemos unidades de diferentes tamaños:

caracteres $\longrightarrow$ subpalabras frecuentes $\longrightarrow$ algunas palabras completas.

Interpretación

BPE aprende el vocabulario **a partir de las frecuencias observadas en el corpus**.

BPE: los merges ocurren dentro de las palabras[1]

En la práctica, BPE no trabaja sobre todo el texto como una única secuencia de caracteres.

Primero se separa el corpus en palabras:

`set ␣new ␣new ␣renew ␣reset ␣renew`

Las frecuencias se calculan utilizando todo el corpus, pero:

los merges no pueden cruzar las fronteras entre palabras.

[1] Jurafsky y Martin, Sec. 2.4.1, ejemplo (2.11).

Ejemplo

Partimos del corpus:

`set ␣new ␣new ␣renew ␣reset ␣renew`

Agrupamos las palabras iguales y contamos sus apariciones:

Frecuencia	Palabra
2	`␣n e w`
2	`␣r e n e w`
1	`s e t`
1	`␣r e s e t`

El vocabulario inicial contiene:

$$V = \{␣, \mathtt{e}, \mathtt{n}, \mathtt{r}, \mathtt{s}, \mathtt{t}, \mathtt{w}\}.$$

¿Qué pares puede considerar BPE?

Por ejemplo, en

$$\texttt{␣n e w}$$

los pares adyacentes son

$$(\texttt{␣}, \texttt{n}), \qquad (\texttt{n}, \texttt{e}), \qquad (\texttt{e}, \texttt{w}).$$

Mientras que en

$$\texttt{␣r e n e w}$$

tenemos

$$(\texttt{␣}, \texttt{r}), \quad (\texttt{r}, \texttt{e}), \quad (\texttt{e}, \texttt{n}), \quad (\texttt{n}, \texttt{e}), \quad (\texttt{e}, \texttt{w}).$$

Pero nunca se consideran pares entre **dos palabras distintas**.

Primer merge

Contamos los pares dentro de las palabras.
El par más frecuente es

$$(\mathtt{n}, \mathtt{e}).$$

Aparece en:

$$\mathtt{␣n\ e\ w} \qquad \text{(frecuencia 2)}$$

y en

$$\mathtt{␣r\ e\ n\ e\ w} \qquad \text{(frecuencia 2)}.$$

Por tanto,

$$\mathtt{count}(\mathtt{n\ e}) = 2 + 2 = 4.$$

Creamos un nuevo token:

$$\boxed{\mathtt{n} + \mathtt{e} \longrightarrow \mathtt{ne}}$$

Corpus después del primer merge

Después de

$$\texttt{n} + \texttt{e} \longrightarrow \texttt{ne},$$

tenemos:

Frecuencia	Palabra
2	␣ne w
2	␣r e ne w
1	s e t
1	␣r e s e t

El nuevo vocabulario contiene ahora el token ne

$$V = \{\text{␣}, \texttt{e}, \texttt{n}, \texttt{r}, \texttt{s}, \texttt{t}, \texttt{w}, \texttt{ne}\}.$$

Segundo merge

Volvemos a contar los pares. Ahora el más frecuente es

$$(\texttt{ne}, \texttt{w}),$$

con frecuencia total

$$4.$$

Por tanto:

$$\boxed{\texttt{ne} + \texttt{w} \longrightarrow \texttt{new}}$$

y obtenemos:

Frecuencia	Palabra
2	␣new
2	␣r e new
1	s e t
1	␣r e s e t

$$V = \{␣, \texttt{e}, \texttt{n}, \texttt{r}, \texttt{s}, \texttt{t}, \texttt{w}, \texttt{ne}, \texttt{new}\}.$$

El espacio también puede formar parte de un token

Después de aprender `new`, el siguiente par más frecuente es

$$(\text{␣}, \texttt{r}),$$

que aparece al inicio de

␣r e new y ␣r e s e t.

Se realiza entonces el merge

$$\boxed{\text{␣} + \texttt{r} \longrightarrow \text{␣}\texttt{r}}$$

El espacio puede formar parte de un token, pero sigue indicando el **inicio de una palabra**.

$$V = \{\text{␣}, \texttt{e}, \texttt{n}, \texttt{r}, \texttt{s}, \texttt{t}, \texttt{w}, \texttt{ne}, \texttt{new}, \text{␣}\texttt{r}\}.$$

Aprendiendo el prefijo re-

El siguiente merge es

$$(\text{␣r}, \text{e}) \longrightarrow \text{␣re}.$$

Entonces obtenemos:

Frecuencia	Palabra
2	␣new
2	␣re new
1	s e t
1	␣re s e t

El sistema ha aprendido esencialmente una unidad

$$\boxed{\text{␣re}}$$

que aparece al comienzo de distintas palabras.

$$V = \{\text{␣}, \text{e}, \text{n}, \text{r}, \text{s}, \text{t}, \text{w}, \text{ne}, \text{new}, \text{␣r}, \text{␣re}\}.$$

Si continuamos...

Si continuamos los merges, obendríamos

$$V = \{\text{␣}, \texttt{e}, \texttt{n}, \texttt{r}, \texttt{s}, \texttt{t}, \texttt{w}, \texttt{ne}, \texttt{new}, \text{␣}\texttt{r}, \text{␣}\texttt{re}, \text{␣}\texttt{new}, \text{␣}\texttt{renew}, \texttt{se}, \texttt{set}\}.$$

La idea importante

El espacio inicial

␣

sí puede participar en los merges de su palabra.
Por ejemplo:

$$\text{␣} + \texttt{r} + \texttt{e} \longrightarrow \text{␣}\texttt{re}.$$

Pero las palabras siguen siendo unidades separadas:

set	␣new	␣renew	␣reset

BPE no realiza merges entre dos palabras distintas.

BPE aprende subpalabras reutilizables

En el ejemplo, BPE aprende progresivamente tokens como

$$\texttt{ne}, \qquad \texttt{new}, \qquad \texttt{␣r}, \qquad \texttt{␣re}.$$

Así,

$$\texttt{␣renew}$$

puede representarse temporalmente como

$$\boxed{\texttt{␣re} + \texttt{new}.}$$

Los conteos provienen del corpus completo, pero

$$\boxed{\text{los merges se restringen al interior de cada palabra.}}$$

BPE encoder

Una vez aprendido el vocabulario, el **BPE encoder** se utiliza para tokenizar texto nuevo.

1. Se separa cada palabra en caracteres.
2. Se aplican los **merges aprendidos durante el entrenamiento**, en el mismo orden en que fueron aprendidos.

Por ejemplo, si durante el entrenamiento aprendimos:

$$\texttt{n} + \texttt{e} \longrightarrow \texttt{ne},$$

$$\texttt{ne} + \texttt{w} \longrightarrow \texttt{new},$$

el encoder aplica primero:

$$\texttt{n} \;\; \texttt{e} \longrightarrow \texttt{ne},$$

y después:

$$\texttt{ne} \;\; \texttt{w} \longrightarrow \texttt{new}.$$

Importante: Las frecuencias del texto nuevo **no intervienen**, se utilizan los merges y su orden aprendidos con el corpus de entrenamiento.

BPE encoder: palabras no vistas

Los merges pueden capturar unidades que aparecen después en **palabras no vistas durante el entrenamiento**.

Por ejemplo, el entrenamiento anterior aprendió:

`␣re`

como una unidad asociada al prefijo `re-`.
Puede reutilizarse posteriormente en palabras como

`␣revisit` o `␣rearrange`.

También aprendió el token `new` sin un espacio inicial, por lo que puede aparecer **dentro de otras palabras**, por ejemplo:

`␣a new`.

Así, las palabras desconocidas pueden representarse mediante **varios tokens conocidos**.

Corpus

Un **corpus** es una colección de textos (o transcripciones de habla) utilizada para estudiar o modelar el lenguaje.

Pero los textos de un corpus no son una muestra neutral del "lenguaje en general". Pueden variar por:

- idioma y variedad lingüística,
- género: noticias, libros, redes sociales, artículos científicos, etc.,
- contexto: conversación, texto editado, notas médicas, documentos legales, etc.,
- características de quienes producen el texto,
- periodo histórico.

Consecuencia para Machine Learning

El corpus de entrenamiento determina qué patrones puede aprender el modelo y qué tan bien podrá generalizar a nuevos textos.

¿De dónde vienen nuestros datos?

Antes de entrenar un modelo con texto conviene preguntar:

- ¿Por qué y para qué se recolectó el corpus?
- ¿Quién produjo los textos y en qué contexto?
- ¿Qué idioma, región o variedad lingüística contiene?
- ¿Cómo se seleccionaron los documentos?
- ¿Qué preprocesamiento se realizó?
- ¿Existen anotaciones? ¿Cómo se obtuvieron?

Se recomienda documentar estas características mediante *datasheets* o *data statements*.

Idea importante

Un cambio entsre la distribución del corpus de entrenamiento y la de los textos reales puede producir una pérdida importante de desempeño.

Preprocesamiento con expresiones regulares

Una **expresión regular** (regex) describe un patrón de caracteres.

En procesamiento de textos puede utilizarse para:

- buscar patrones,
- identificar determinados tipos de cadenas,
- sustituir o normalizar fragmentos de texto,
- apoyar la etapa de **pre-tokenización**.

Preprocesamiento con expresiones regulares

Una **expresión regular** (regex) describe un patrón de caracteres.

En procesamiento de textos puede utilizarse para:

- buscar patrones,
- identificar determinados tipos de cadenas,
- sustituir o normalizar fragmentos de texto,
- apoyar la etapa de **pre-tokenización**.

Por ejemplo, en Python:

```
re.search(pattern, string)
```

busca la primera coincidencia del patrón dentro de una cadena.

Preprocesamiento con expresiones regulares

Una **expresión regular** (regex) describe un patrón de caracteres.
En procesamiento de textos puede utilizarse para:

- buscar patrones,
- identificar determinados tipos de cadenas,
- sustituir o normalizar fragmentos de texto,
- apoyar la etapa de **pre-tokenización**.

Por ejemplo, en Python:

```
re.search(pattern, string)
```

busca la primera coincidencia del patrón dentro de una cadena.

En este curso

No estudiaremos regex exhaustivamente. Nos interesan como una herramienta práctica para preparar texto antes de construir las representaciones numéricas.

Expresiones regulares: símbolos comunes

Símbolo	Significado	Ejemplo
`\d`	Cualquier dígito (0–9)	`r"\d"`
`\w`	Letra, dígito o _	`r"\w"`
`\s`	Espacio en blanco	`r"\s"`
`.`	Cualquier carácter	`r”a.b"`
`+`	Una o más repeticiones	`r"\d+"`
`*`	Cero o más repeticiones	`r"\d*"`
`?`	Cero o una repetición	`r"https?"`
`[...]`	Uno de los caracteres indicados	`r"[abc]"`
`[A-Z]`	Un carácter dentro del rango	`r"[A-Z]"`
`^`	Inicio del texto	`r"^Hola"`
`$`	Final del texto	`r"fin$"`
`\`	Escapa un símbolo especial	`r"\."`

Ejemplo [2]

En Python, las expresiones regulares se utilizan mediante el módulo `re`:

```
import re
```

Por ejemplo:

```
re.findall(r"\d+", "Tengo 3 libros y 12 revistas")
# ['3', '12']
```

Aquí, `\d` significa "dígito" y `+` significa "uno o más", por lo que `\d+` busca **uno o más dígitos consecutivos**.

[2] Mas detalles en *Speech and Language Processing - Jurafsky, Martin*, Sec. 2.6.

Expresiones regulares: ejemplos comunes en Python

Patrón	Ejemplo de uso
`r"\d+"`	Uno o más dígitos
`r"[A-Za-z]+"`	Una o más letras
`r"\s+"`	Uno o más espacios en blanco
`r"https?://\S+"`	Una URL que comienza con `http://` o `https://`
`r"\w+@\w+\.\w+"`	Patrón simple para un correo electrónico
`r"^Hola"`	Texto que comienza con `Hola`
`r"fin$"`	Texto que termina con `fin`

Por ejemplo, para eliminar números de un texto:

```
texto = "Tengo 3 libros y 12 revistas"
resultado = re.sub(r"\d+", "", texto)
```

¿Qué significa “limpiar” un texto?

Dependiendo del problema, el preprocesamiento puede incluir decisiones como:

- convertir o no a minúsculas,
- tratar signos de puntuación,
- normalizar espacios,
- identificar URLs, números u otros patrones,
- separar el texto en unidades apropiadas para la tarea.

¿Qué significa “limpiar” un texto?

Dependiendo del problema, el preprocesamiento puede incluir decisiones como:

- convertir o no a minúsculas,
- tratar signos de puntuación,
- normalizar espacios,
- identificar URLs, números u otros patrones,
- separar el texto en unidades apropiadas para la tarea.

Ejemplo:

```
¡Machine Learning es GENIAL!!!
```

podría transformarse en

```
machine learning es genial
```

¿Qué significa “limpiar” un texto?

Dependiendo del problema, el preprocesamiento puede incluir decisiones como:

- convertir o no a minúsculas,
- tratar signos de puntuación,
- normalizar espacios,
- identificar URLs, números u otros patrones,
- separar el texto en unidades apropiadas para la tarea.

Ejemplo:

```
¡Machine Learning es GENIAL!!!
```

podría transformarse en

```
machine learning es genial
```

Cuidado

No existe un preprocesamiento universalmente correcto. Mayúsculas, puntuación y otros símbolos pueden contener información útil para ciertas tareas.

Del texto a los datos

Al terminar esta etapa tenemos una secuencia de unidades discretas:

"El modelo aprende del texto"

$\Downarrow$

$[\texttt{el}, \texttt{modelo}, \texttt{aprende}, \texttt{del}, \texttt{texto}]$

Del texto a los datos

Al terminar esta etapa tenemos una secuencia de unidades discretas:

"El modelo aprende del texto"

$$\Downarrow$$

$$[\texttt{el}, \texttt{modelo}, \texttt{aprende}, \texttt{del}, \texttt{texto}]$$

Pero los algoritmos de Machine Learning todavía necesitan números:

$$[t_1, t_2, \ldots, t_n] \quad \longrightarrow \quad x \in \mathbb{R}^p.$$

4.2 Representación numérica de textos[3]

Hasta ahora tenemos documentos formados por palabras o tokens.

Pero los algoritmos de aprendizaje de máquina trabajan con **vectores numéricos**:

$$\boxed{\text{Documento} \quad \longrightarrow \quad x \in \mathbb{R}^p}$$

La pregunta central es:

¿Cómo convertimos un texto en un vector?

[3] Jurafsky y Martin, Secs. 11.1.1–11.1.3 y 5.2–5.5.

Texto como conjunto de características

Supongamos que tenemos los documentos

d_1 : "machine learning is useful",
d_2 : "machine learning uses data",
d_3 : "data science is useful".

Podemos construir un vocabulario

$$V = \{\text{machine}, \text{learning}, \text{is}, \text{useful}, \text{uses}, \text{data}, \text{science}\}.$$

Cada palabra puede convertirse en una **característica**.

Bag of Words

Una representación sencilla es el modelo

Bag of Words

Cada dimensión del vector corresponde a una palabra del vocabulario.

Si

$$V = [\text{machine}, \text{learning}, \text{is}, \text{useful}, \text{data}],$$

entonces

"machine learning is useful"

puede representarse como

$$x = [1, 1, 1, 1, 0].$$

¿Por qué “Bag of Words”?

El documento

“machine learning uses data”

y cualquier reordenamiento de las mismas palabras, por ejemplo,

“data uses machine learning”,

tienen la misma representación.

Bag of Words ignora el **orden** de las palabras.

Solo conserva información sobre

qué palabras aparecen y cuántas veces

Conteos de palabras

Consideremos

$$V = [\text{machine}, \text{learning}, \text{data}].$$

Para el documento

"machine learning learning data",

la representación mediante conteos es

$$x = [1, 2, 1].$$

En general,

$x_j =$ número de veces que aparece la palabra j en el documento.

Matriz documento–término

Para varios documentos podemos construir una matriz:

$$X = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{bmatrix}.$$

	machine	learning	is	useful	data
d_1	1	1	1	1	0
d_2	1	1	0	0	1
d_3	0	0	1	1	1

Es simplemente nuestra matriz habitual de aprendizaje de máquina:

filas = documentos, columnas = características

Una característica importante: dispersión

En un corpus real, el vocabulario puede contener miles o decenas de miles de palabras.

Sin embargo, cada documento utiliza solo una pequeña fracción del vocabulario.

$$x = [0, 0, 3, 0, 0, 1, 0, \ldots, 0]$$

Por tanto, los vectores Bag of Words suelen ser

muy largos y dispersos

(*sparse vectors*).

¿Todas las palabras deberían pesar igual?

Supongamos que una palabra aparece muchas veces en **todos** los documentos.

¿Es realmente útil para distinguirlos?

Por ejemplo:

the, is, of, and, . . .

Queremos asignar mayor importancia a términos que

aparecen en un documento	pero no en todos los documentos

Esto motiva la representación

tf-idf

Term Frequency (TF)

La primera parte mide qué tan frecuente es una palabra dentro de un documento.

$$\mathrm{tf}_{t,d}$$

Una opción sencilla sería usar directamente la frecuencia de la palabra t en el documento d

$$\mathrm{tf}_{t,d} = \mathrm{count}(t, d).$$

Pero se utiliza una transformación logarítmica:

$$\mathrm{tf}_{t,d} = \begin{cases} 1 + \log_{10} \mathrm{count}(t, d), & \mathrm{count}(t, d) > 0, \\ 0, & \text{en otro caso.} \end{cases}$$

Inverse Document Frequency (IDF)

Sea $N =$ número total de documentos y
$df_t =$ número de documentos que contienen el término t.
Entonces

$$\boxed{\text{idf}_t = \log_{10}\left(\frac{N}{df_t}\right)}$$

- Si t aparece en muchos documentos:

$$df_t \text{ grande} \quad \Rightarrow \quad idf_t \text{ pequeño.}$$

- Si t aparece en pocos documentos:

$$df_t \text{ pequeño} \quad \Rightarrow \quad idf_t \text{ grande.}$$

Ejemplo de IDF

Supongamos que tenemos

$$N = 1000$$

documentos.

Si una palabra aparece en 500 documentos:

$$idf = \log_{10}\left(\frac{1000}{500}\right) = \log_{10}(2).$$

Pero si aparece solamente en 10 documentos:

$$idf = \log_{10}\left(\frac{1000}{10}\right) = 2.$$

La segunda palabra recibe un peso mucho mayor.

Ejemplo de valores IDF

A continuación se muestran algunos valores de *idf* para palabras del corpus de obras de Shakespeare.

Las palabras que aparecen únicamente en una obra, como *Romeo*, tienen un valor de *idf* relativamente alto, mientras que palabras que aparecen en muchas obras, como *fool*, tienen valores pequeños.

Si una palabra aparece en todas las obras, como *good* o *sweet*, su valor de *idf* es 0 y por lo tanto no ayuda a discriminar entre documentos.

Palabra	**df**	**idf**
Romeo	1	1.57
salad	2	1.27
Falstaff	4	0.967
forest	12	0.489
battle	21	0.246
wit	34	0.037
fool	36	0.012
good	37	0
sweet	37	0

TF-IDF

Combinamos ambas cantidades:

$$\boxed{\text{tf-idf}(t, d) = \text{tf}_{t,d}\, \text{idf}_t}$$

La representación de un documento se convierte entonces en

$$x_d = [\text{tf-idf}(t_1, d), \ldots, \text{tf-idf}(t_p, d)].$$

Una palabra recibe un peso grande cuando

es frecuente en el documento y poco frecuente en la colección

Comparando documentos

Una vez que cada documento es un vector,

$$d_1 \longrightarrow x_1, \qquad d_2 \longrightarrow x_2,$$

podemos medir qué tan similares son.

Una de las medidas más utilizadas es

$$\boxed{\cos(x_1, x_2) = \frac{x_1^\top x_2}{\|x_1\|\|x_2\|}}$$

Es decir, el producto punto **normalizado por la longitud de los vectores**.

Interpretación geométrica del coseno

$$\cos(\theta) = \frac{x^\top y}{\|x\|\|y\|}$$

Para vectores de conteos o TF-IDF:

$$0 \leq \cos(\theta) \leq 1.$$

- $\cos(\theta) \approx 1$: vectores en direcciones muy parecidas.
- $\cos(\theta) \approx 0$: vectores casi ortogonales.

Mayor coseno $\Rightarrow$ mayor similitud.

¿Por qué normalizar?

Consideremos

$$x = [1, 1], \qquad y = [10, 10].$$

Aunque los vectores tienen magnitudes muy distintas,

$$\cos(x, y) = 1.$$

Ambos apuntan exactamente en la misma dirección.

Por eso el coseno compara principalmente el **patrón relativo de términos**, no simplemente la longitud del documento.

Ejemplo: búsqueda con TF-IDF

Queremos encontrar qué documentos son más relevantes para la consulta

Query: sweet love

en una colección de cuatro documentos:

d_1 : sweet sweet nurse love

d_2 : sweet sorrow

d_3 : how sweet is love

d_4 : nurse

[4] Conserva en el pizarrón $d_1, ..., d_4$ y $\mathcal{V}$.

Ejemplo: búsqueda con TF-IDF

Queremos encontrar qué documentos son más relevantes para la consulta

Query: sweet love

en una colección de cuatro documentos:

d_1 : sweet sweet nurse love

d_2 : sweet sorrow

d_3 : how sweet is love

d_4 : nurse

El vocabulario de la colección[4] es

$$\mathcal{V} = \{\text{sweet}, \text{nurse}, \text{love}, \text{how}, \text{sorrow}, \text{is}\}.$$

Compararemos la consulta con cada documento utilizando

$$\text{score}(q, d) = \cos(q, d) = \frac{q \cdot d}{\|q\| \, \|d\|}$$

~~donde los vectores contienen los pesos~~ **TF-IDF**.

[4] Conserva en el pizarrón $d_1, ..., d_4$ y $\mathcal{V}$.

Paso 1: calcular IDF

Tenemos $N = 4$ documentos. Para cada término,

$$\text{idf}(t) = \log_{10}\left(\frac{N}{df_t}\right).$$

Término	df_t	$idf(t)$
sweet		
nurse		
love		
how		
sorrow		
is		

Paso 1: calcular IDF

Tenemos $N = 4$ documentos. Para cada término,

$$\text{idf}(t) = \log_{10}\left(\frac{N}{df_t}\right).$$

Término	df_t	$idf(t)$
sweet	3	$\log_{10}(4/3) = 0{,}125$
nurse	2	$\log_{10}(4/2) = 0{,}301$
love	2	$\log_{10}(4/2) = 0{,}301$
how	1	$\log_{10}(4) = 0{,}602$
sorrow	1	$\log_{10}(4) = 0{,}602$
is	1	$\log_{10}(4) = 0{,}602$

Paso 1: calcular IDF

Tenemos $N = 4$ documentos. Para cada término,

$$\text{idf}(t) = \log_{10}\left(\frac{N}{df_t}\right).$$

Término	df_t	$idf(t)$
sweet	3	$\log_{10}(4/3) = 0{,}125$
nurse	2	$\log_{10}(4/2) = 0{,}301$
love	2	$\log_{10}(4/2) = 0{,}301$
how	1	$\log_{10}(4) = 0{,}602$
sorrow	1	$\log_{10}(4) = 0{,}602$
is	1	$\log_{10}(4) = 0{,}602$

Interpretación

Los términos que aparecen en menos documentos reciben un mayor peso IDF. Por ejemplo, *sorrow* aparece solamente en un documento, mientras que *sweet* aparece en tres de los cuatro documentos.

Paso 2: vector TF-IDF de la consulta

La consulta es

$$q = \text{“sweet love”}.$$

Recordando la definición de tf

$$\text{tf}(t,d) = \begin{cases} 1 + \log_{10}(\text{count}(t,d)), & \text{count} > 0, \\ 0, & \text{en otro caso.} \end{cases}$$

Como *sweet* y *love* aparecen una vez:

$$tf(\text{sweet}, q) = tf(\text{love}, q) = 1.$$

Paso 2: vector TF-IDF de la consulta

Por tanto,

$$q = \begin{bmatrix} \mathrm{tf}(t_1, q)\,\mathrm{idf}(t_1) & \mathrm{tf}(t_2, q)\,\mathrm{idf}(t_2) & \cdots & \mathrm{tf}(t_p, q)\,\mathrm{idf}(t_p) \end{bmatrix}.$$

$$q =$$

Paso 2: vector TF-IDF de la consulta

Por tanto,

$$q = \begin{bmatrix} \mathrm{tf}(t_1, q)\,\mathrm{idf}(t_1) & \mathrm{tf}(t_2, q)\,\mathrm{idf}(t_2) & \cdots & \mathrm{tf}(t_p, q)\,\mathrm{idf}(t_p) \end{bmatrix}.$$

$$q = \begin{bmatrix} 0{,}125 & 0 & 0{,}301 & 0 & 0 & 0 \end{bmatrix}.$$

Su norma es

Paso 2: vector TF-IDF de la consulta

Por tanto,

$$q = \begin{bmatrix} \mathrm{tf}(t_1, q)\,\mathrm{idf}(t_1) & \mathrm{tf}(t_2, q)\,\mathrm{idf}(t_2) & \cdots & \mathrm{tf}(t_p, q)\,\mathrm{idf}(t_p) \end{bmatrix}.$$

$$q = \begin{bmatrix} 0{,}125 & 0 & 0{,}301 & 0 & 0 & 0 \end{bmatrix}.$$

Su norma es

$$\|q\| = \sqrt{0{,}125^2 + 0{,}301^2} = 0{,}326.$$

Al normalizar:

$$\boxed{\hat{q} = \begin{bmatrix} 0{,}383 & 0 & 0{,}924 & 0 & 0 & 0 \end{bmatrix}}$$

Paso 3: TF-IDF del Documento 1

$$d_1 = \text{“sweet sweet nurse love”}$$

La palabra *sweet* aparece dos veces:

$$tf(\text{sweet}, d_1) = 1 + \log_{10}(2) = 1{,}301.$$

Por tanto:

Término	*tf*	*idf*	*tf-idf*
sweet	1,301	0,125	0,163
nurse	1	0,301	0,301
love	1	0,301	0,301

Paso 3: TF-IDF del Documento 1

$$d_1 = \text{“sweet sweet nurse love”}$$

La palabra *sweet* aparece dos veces:

$$tf(\text{sweet}, d_1) = 1 + \log_{10}(2) = 1{,}301.$$

Por tanto:

Término	*tf*	*idf*	*tf-idf*
sweet	1,301	0,125	0,163
nurse	1	0,301	0,301
love	1	0,301	0,301

$$\|d_1\| = \sqrt{0{,}163^2 + 0{,}301^2 + 0{,}301^2} = 0{,}456.$$

Entonces el vector normalizado es

$$\hat{d}_1 = \begin{bmatrix} 0{,}357 & 0{,}661 & 0{,}661 & 0 & 0 & 0 \end{bmatrix}.$$

Paso 4: similitud con el Documento 1

Recordemos:

$$\hat{q} = \begin{bmatrix} 0{,}383 & 0 & 0{,}924 & 0 & 0 & 0 \end{bmatrix}$$

y

$$\hat{d}_1 = \begin{bmatrix} 0{,}357 & 0{,}661 & 0{,}661 & 0 & 0 & 0 \end{bmatrix}.$$

Paso 4: similitud con el Documento 1

Recordemos:

$$\hat{q} = \begin{bmatrix} 0{,}383 & 0 & 0{,}924 & 0 & 0 & 0 \end{bmatrix}$$

y

$$\hat{d}_1 = \begin{bmatrix} 0{,}357 & 0{,}661 & 0{,}661 & 0 & 0 & 0 \end{bmatrix}.$$

La similitud coseno es simplemente el producto punto:

$$\begin{aligned} \cos(q, d_1) &= \hat{q} \cdot \hat{d}_1 \\ &= (0{,}383)(0{,}357) + (0)(0{,}661) + (0{,}924)(0{,}661) \\ &= 0{,}137 + 0{,}610 \\ &= \boxed{0{,}747}. \end{aligned}$$

Observación

Solo contribuyen al producto punto los términos que aparecen **tanto en la consulta como en el documento**.

Paso 5: similitud con el Documento 2

$$d_2 = \text{“sweet sorrow”}.$$

Su vector TF-IDF es

$$d_2 = \begin{bmatrix} 0{,}125 & 0 & 0 & 0 & 0{,}602 & 0 \end{bmatrix}.$$

Su norma:

$$\|d_2\| = \sqrt{0{,}125^2 + 0{,}602^2} = 0{,}615.$$

Por tanto,

$$\hat{d}_2 = \begin{bmatrix} 0{,}203 & 0 & 0 & 0 & 0{,}979 & 0 \end{bmatrix}.$$

Paso 5: similitud con el Documento 2

$$d_2 = \text{“sweet sorrow”}.$$

Su vector TF-IDF es

$$d_2 = \begin{bmatrix} 0{,}125 & 0 & 0 & 0 & 0{,}602 & 0 \end{bmatrix}.$$

Su norma:

$$\|d_2\| = \sqrt{0{,}125^2 + 0{,}602^2} = 0{,}615.$$

Por tanto,

$$\hat{d}_2 = \begin{bmatrix} 0{,}203 & 0 & 0 & 0 & 0{,}979 & 0 \end{bmatrix}.$$

Como la consulta solo comparte *sweet* con d_2:

$$\cos(q, d_2) = (0{,}383)(0{,}203) = \boxed{0{,}0779}.$$

Resultado: ¿qué documento es más relevante?

Para los dos documentos calculados:

$$\boxed{\cos(q, d_1) = 0{,}747 \qquad \cos(q, d_2) = 0{,}0779}$$

Por lo tanto,

$$\boxed{d_1 > d_2}$$

para la consulta *"sweet love"*.

¿Por qué?

- d_1 contiene **ambos términos** de la consulta: *sweet* y *love*.
- Además, *sweet* aparece dos veces en d_1.
- d_2 solamente contiene *sweet*.
- *sorrow* tiene un IDF alto, pero no aparece en la consulta, por lo que **no contribuye al producto punto**.

Resultado: ¿qué documento es más relevante?

Para los dos documentos calculados:

$$\boxed{\cos(q, d_1) = 0{,}747 \qquad \cos(q, d_2) = 0{,}0779}$$

Por lo tanto,

$$\boxed{d_1 > d_2}$$

para la consulta *"sweet love"*.

¿Por qué?

- d_1 contiene **ambos términos** de la consulta: *sweet* y *love*.
- Además, *sweet* aparece dos veces en d_1.
- d_2 solamente contiene *sweet*.
- *sorrow* tiene un IDF alto, pero no aparece en la consulta, por lo que **no contribuye al producto punto**.

Idea clave

TF-IDF determina el peso de los términos; la similitud coseno utiliza esos pesos para medir qué tan alineados están el documento y la consulta.

Hasta ahora: representaciones dispersas

Partimos de un **corpus de entrenamiento**:

$$\boxed{\mathcal{D} = \{d_1, d_2, \ldots, d_N\} \longrightarrow \text{Vocabulario } \mathcal{V} = \{t_1, t_2, \ldots, t_p\}}$$

Hasta ahora: representaciones dispersas

Partimos de un **corpus de entrenamiento**:

$$\boxed{\mathcal{D} = \{d_1, d_2, \ldots, d_N\} \longrightarrow \text{Vocabulario } \mathcal{V} = \{t_1, t_2, \ldots, t_p\}}$$

El vocabulario define las p dimensiones con las que representamos cada documento:

$$d_i \longrightarrow x_i = (x_{i1}, x_{i2}, \ldots, x_{ip}),$$

donde podemos elegir, por ejemplo,

$$\boxed{x_{ij} = \begin{cases} \text{count}(t_j, d_i), & \text{Bag of Words,} \\ \text{tf}(t_j, d_i)\,\text{idf}(t_j), & \text{TF-IDF.} \end{cases}}$$

Una vez construido el vector podemos:

comparar	clasificar
$\cos(x_i, x_j)$	$f(x_i)$

agrupar
K-means

El vocabulario y los valores IDF se obtienen del **corpus de entrenamiento**.

Otra idea: representar significado con vectores

Hasta ahora las dimensiones del vector corresponden directamente a palabras del vocabulario.

Pero también podemos intentar que la posición de un vector represente algo sobre su **significado**.

La idea fundamental es:

palabras que aparecen en contextos similares $\Rightarrow$ significados relacionados

Esta es la base de la llamada

semántica vectorial.

Vectores basados en contexto

En lugar de representar un **documento**, podemos representar una **palabra**.

Por ejemplo:

	pie	data	computer
cherry	442	8	2
digital	5	1683	1670
information	5	3982	3325

Cada número indica cuántas veces la palabra objetivo aparece cerca de una palabra de contexto.

Contextos similares ⇒ vectores similares

Usando los vectores anteriores:

$$\cos(\text{cherry}, \text{information}) \approx 0{,}018,$$

mientras que

$$\cos(\text{digital}, \text{information}) \approx 0{,}996.$$

Por tanto,

digital está mucho más cerca de information

porque ambas palabras aparecen en contextos parecidos.

De vectores dispersos a embeddings

Las representaciones basadas directamente en conteos pueden tener dimensión

$$|V|,$$

donde $|V|$ es el tamaño del vocabulario.

Esto produce vectores:

largos	y	dispersos

Los **embeddings** utilizan en cambio vectores

$$x \in \mathbb{R}^d, \qquad d \ll |V|,$$

que son

cortos y densos.

¿Qué es un embedding?

Un embedding representa una palabra mediante un vector como

$$\text{learning} \longrightarrow \begin{bmatrix} 0{,}21 \\ -0{,}72 \\ 0{,}18 \\ \vdots \\ 0{,}34 \end{bmatrix} \in \mathbb{R}^d.$$

A diferencia de Bag of Words:

- las dimensiones ya no corresponden directamente a palabras;
- los valores son números reales;
- pueden ser positivos o negativos;
- el vector es denso.

Representaciones dispersas y densas

	Bag of Words / TF-IDF	**Embeddings**
Dimensión	$\approx \lvert V \rvert$	$d \ll \lvert V \rvert$
Representación	Dispersa	Densa
Valores	Conteos o pesos	Números reales
Dimensiones	Interpretables	Generalmente no interpretables
Similitud	Coseno	Coseno

Ambos enfoques convierten texto en

vectores numéricos

pero lo hacen de maneras muy diferentes.

Word2Vec: idea general

Word2Vec aprende automáticamente embeddings a partir de los **contextos** en los que aparecen las palabras.

La idea general es:

Corpus $\longrightarrow$ contextos observados $\longrightarrow$ vectores aprendidos

Palabras utilizadas en contextos similares terminan con vectores cercanos en el espacio.

El significado aparece como una **relación geométrica**.

Word2Vec: visualización

https://projector.tensorflow.org/

Word2Vec: idea general

Word2Vec aprende automáticamente embeddings a partir de los **contextos** en los que aparecen las palabras.

La idea general es:

Corpus $\longrightarrow$ contextos observados $\longrightarrow$ vectores aprendidos

Palabras utilizadas en contextos similares terminan con vectores cercanos en el espacio.

El significado aparece como una **relación geométrica**.

Word2Vec: visualización

https://projector.tensorflow.org/

¿Por qué utilizar embeddings densos?

Una representación dispersa trata dimensiones distintas como independientes.

Por ejemplo,

car y *automobile*

corresponden a dimensiones diferentes del vocabulario.

¿Por qué utilizar embeddings densos?

Una representación dispersa trata dimensiones distintas como independientes.

Por ejemplo,

car y *automobile*

corresponden a dimensiones diferentes del vocabulario.
Aunque sean palabras semánticamente similares, esta relación **no está explícitamente representada**.

Con embeddings densos buscamos que

$$\boxed{w_{\text{car}} \approx w_{\text{automobile}}}$$

y, por tanto,

$$\cos(w_{\text{car}}, w_{\text{automobile}}) \approx 1.$$

¿Por qué utilizar embeddings densos?

Una representación dispersa trata dimensiones distintas como independientes.

Por ejemplo,

$$\textit{car} \qquad \text{y} \qquad \textit{automobile}$$

corresponden a dimensiones diferentes del vocabulario.
Aunque sean palabras semánticamente similares, esta relación **no está explícitamente representada**.

Con embeddings densos buscamos que

$$\boxed{w_{\text{car}} \approx w_{\text{automobile}}}$$

y, por tanto,

$$\cos(w_{\text{car}}, w_{\text{automobile}}) \approx 1.$$

Además, trabajar con $d \ll |V|$ reduce considerablemente la dimensionalidad de la representación.

Word2Vec: aprender mediante una tarea auxiliar

La idea central de Word2Vec:

No aprendemos directamente “el significado” de una palabra.

Word2Vec: aprender mediante una tarea auxiliar

La idea central de Word2Vec:

No aprendemos directamente “el significado” de una palabra.

En su lugar, construimos una **tarea de predicción** utilizando el propio corpus:

> ¿Es probable que la palabra *c* aparezca cerca de la palabra *w*?

Por ejemplo:

apricot y *jam*

probablemente aparecen juntas con mayor frecuencia que

apricot y *bed*.

Word2Vec: aprender mediante una tarea auxiliar

La idea central de Word2Vec:

No aprendemos directamente “el significado” de una palabra.

En su lugar, construimos una **tarea de predicción** utilizando el propio corpus:

> ¿Es probable que la palabra *c* aparezca cerca de la palabra *w*?

Por ejemplo:

apricot y *jam*

probablemente aparecen juntas con mayor frecuencia que

apricot y *bed*.

Los parámetros aprendidos para resolver esta tarea se convierten en los **embeddings**.

Aprendizaje autosupervisado

Consideremos una oración del corpus:

. . . lemon, a [tablespoon of **apricot** jam, a] pinch . . .

Usando una ventana de contexto de ±2:

tablespoon	of	apricot	jam	a
c_1	c_2	w	c_3	c_4

Aprendizaje autosupervisado

Consideremos una oración del corpus:

. . . lemon, a [tablespoon of **apricot** jam, a] pinch . . .

Usando una ventana de contexto de ±2:

tablespoon	of	apricot	jam	a
c_1	c_2	w	c_3	c_4

El propio texto nos proporciona ejemplos positivos:

(apricot, tablespoon), (apricot, of), (apricot, jam), (apricot, a).

No necesitamos etiquetar manualmente los datos:

el corpus proporciona la supervisión

A esta idea se le llama **aprendizaje autosupervisado**.

Skip-gram: ejemplos positivos y negativos

Para entrenar el clasificador necesitamos dos tipos de ejemplos.

Ejemplos positivos

Palabras que realmente aparecen cerca:

w	c_{pos}
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

Ejemplos negativos

Palabras seleccionadas aleatoriamente:

w	c_{neg}
apricot	aardvark
apricot	seven
apricot	forever
apricot	coaxial

Skip-gram: ejemplos positivos y negativos

Para entrenar el clasificador necesitamos dos tipos de ejemplos.

Ejemplos positivos
Palabras que realmente aparecen cerca:

w	c_{pos}
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

Ejemplos negativos
Palabras seleccionadas aleatoriamente:

w	c_{neg}
apricot	aardvark
apricot	seven
apricot	forever
apricot	coaxial

El objetivo es aprender a distinguir

contextos reales vs. contextos aleatorios

Skip-gram: idea general

La intuición de **skip-gram** puede resumirse en cuatro pasos:

1. Considerar una **palabra objetivo** y una palabra vecina de su contexto como un **ejemplo positivo**.

2. Seleccionar aleatoriamente otras palabras del vocabulario para generar **ejemplos negativos**.

3. Utilizar **regresión logística** para entrenar un clasificador que distinga entre ejemplos positivos y negativos.

4. Utilizar los **pesos aprendidos** por el modelo como los **embeddings** de las palabras.

Skip-gram: el problema de clasificación

Dada una palabra objetivo w y una posible palabra de contexto c, queremos estimar

$$\boxed{P(+ \mid w, c)}$$

es decir,

$$P(c \text{ es un contexto real de } w).$$

Skip-gram: el problema de clasificación

Dada una palabra objetivo w y una posible palabra de contexto c, queremos estimar

$$\boxed{P(+ \mid w, c)}$$

es decir,

$$P(c \text{ es un contexto real de } w).$$

La idea es que si dos palabras aparecen juntas frecuentemente, sus embeddings deberían ser similares.

$$\text{sim}(w, c) \approx c^T w.$$

Por tanto:

$$\begin{aligned} c^T w \text{ grande} &\Rightarrow \text{palabras compatibles,} \\ c^T w \text{ pequeño} &\Rightarrow \text{palabras poco compatibles.} \end{aligned}$$

Del producto punto a una probabilidad

El producto punto

$$c^T w$$

puede tomar cualquier valor entre $-\infty$ y ∞, por lo que no puede interpretarse directamente como una probabilidad.

Del producto punto a una probabilidad

El producto punto

$$c^T w$$

puede tomar cualquier valor entre $-\infty$ y ∞, por lo que no puede interpretarse directamente como una probabilidad.
Aplicamos la función sigmoide:

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

Entonces,

$$\boxed{P(+ \mid w, c) = \sigma(c^T w) = \frac{1}{1 + e^{-c^T w}}}$$

y

$$\boxed{P(- \mid w, c) = 1 - P(+ \mid w, c) = \sigma(-c^T w).}$$

Del producto punto a una probabilidad

El producto punto

$$c^T w$$

puede tomar cualquier valor entre $-\infty$ y ∞, por lo que no puede interpretarse directamente como una probabilidad.
Aplicamos la función sigmoide:

$$\sigma(x) = \frac{1}{1 + e^{-x}}.$$

Entonces,

$$\boxed{P(+ \mid w, c) = \sigma(c^T w) = \frac{1}{1 + e^{-c^T w}}}$$

y

$$\boxed{P(- \mid w, c) = 1 - P(+ \mid w, c) = \sigma(-c^T w).}$$

Por tanto, el modelo utiliza la **similitud entre embeddings** para decidir si dos palabras forman un contexto plausible.

Skip-gram: múltiples palabras de contexto

Hasta ahora consideramos la probabilidad para una sola palabra de contexto:

$$P(+ \mid w, c_i) = \sigma(c_i^T w).$$

Skip-gram: múltiples palabras de contexto

Hasta ahora consideramos la probabilidad para una sola palabra de contexto:

$$P(+ \mid w, c_i) = \sigma(c_i^T w).$$

Sin embargo, una palabra objetivo w tiene varias palabras en su ventana de contexto:

$$c_1, c_2, \ldots, c_L.$$

Skip-gram hace la suposición simplificadora de que las palabras de contexto son **independientes**. Por tanto,

$$\boxed{P(+ \mid w, c_{1:L}) = \prod_{i=1}^{L} \sigma(c_i^T w)}$$

Skip-gram: múltiples palabras de contexto

Hasta ahora consideramos la probabilidad para una sola palabra de contexto:

$$P(+ \mid w, c_i) = \sigma(c_i^T w).$$

Sin embargo, una palabra objetivo w tiene varias palabras en su ventana de contexto:

$$c_1, c_2, \ldots, c_L.$$

Skip-gram hace la suposición simplificadora de que las palabras de contexto son **independientes**. Por tanto,

$$\boxed{P(+ \mid w, c_{1:L}) = \prod_{i=1}^{L} \sigma(c_i^T w)}$$

Tomando logaritmo, el producto se convierte en una suma:

$$\boxed{\log P(+ \mid w, c_{1:L}) = \sum_{i=1}^{L} \log \sigma(c_i^T w)}$$

Skip-gram: múltiples palabras de contexto

Hasta ahora consideramos la probabilidad para una sola palabra de contexto:

$$P(+ \mid w, c_i) = \sigma(c_i^T w).$$

Sin embargo, una palabra objetivo w tiene varias palabras en su ventana de contexto:

$$c_1, c_2, \ldots, c_L.$$

Skip-gram hace la suposición simplificadora de que las palabras de contexto son **independientes**. Por tanto,

$$\boxed{P(+ \mid w, c_{1:L}) = \prod_{i=1}^{L} \sigma(c_i^T w)}$$

Tomando logaritmo, el producto se convierte en una suma:

$$\boxed{\log P(+ \mid w, c_{1:L}) = \sum_{i=1}^{L} \log \sigma(c_i^T w)}$$

Así, la probabilidad del contexto depende de la **similitud entre el**

¿Qué está aprendiendo Word2Vec?

Skip-gram mantiene realmente **dos vectores por palabra**:

$$\boxed{w = \text{embedding como palabra objetivo}}$$

y

$$\boxed{c = \text{embedding como palabra de contexto.}}$$

¿Qué está aprendiendo Word2Vec?

Skip-gram mantiene realmente **dos vectores por palabra**:

$$\boxed{w = \text{embedding como palabra objetivo}}$$

y

$$\boxed{c = \text{embedding como palabra de contexto.}}$$

Por tanto, se aprenden dos matrices:

$$W \in \mathbb{R}^{|V| \times d}, \qquad C \in \mathbb{R}^{|V| \times d}.$$

W : embeddings de palabras objetivo
C : embeddings de palabras de contexto

Los valores de estas matrices son los **parámetros que se ajustan durante el entrenamiento**.

¿Cómo se aprenden los embeddings?

Inicialmente, los embeddings contienen valores aleatorios.

$$W, C \quad \longleftarrow \quad \text{valores aleatorios.}$$

¿Cómo se aprenden los embeddings?

Inicialmente, los embeddings contienen valores aleatorios.

$$W, C \quad \longleftarrow \quad \text{valores aleatorios.}$$

Durante el entrenamiento queremos:

(w, c_{pos}) :	$w^T c_{\text{pos}}$	aumenta,
(w, c_{neg}) :	$w^T c_{\text{neg}}$	disminuye.

Geométricamente:

$$\text{apricot} \quad \underset{\text{acercar}}{\longleftrightarrow} \quad \text{jam}$$

mientras que

$$\text{apricot} \quad \underset{\text{separar}}{\longleftarrow \quad \longrightarrow} \quad \text{aardvark}$$

Negative sampling

Para cada par positivo

$$(w, c_{\text{pos}})$$

se generan k pares negativos:

$$(w, c_{\text{neg},1}), \ldots, (w, c_{\text{neg},k}).$$

Negative sampling

Para cada par positivo

$$(w, c_{\text{pos}})$$

se generan k pares negativos:

$$(w, c_{\text{neg},1}), \ldots, (w, c_{\text{neg},k}).$$

Las palabras negativas se muestrean del vocabulario.
El libro utiliza una distribución modificada:

$$P_\alpha(w) = \frac{\text{count}(w)^\alpha}{\sum_{w'} \text{count}(w')^\alpha},$$

donde normalmente

$$\boxed{\alpha = 0{,}75}.$$

Negative sampling

Para cada par positivo

$$(w, c_{\text{pos}})$$

se generan k pares negativos:

$$(w, c_{\text{neg},1}), \ldots, (w, c_{\text{neg},k}).$$

Las palabras negativas se muestrean del vocabulario.
El libro utiliza una distribución modificada:

$$P_\alpha(w) = \frac{\text{count}(w)^\alpha}{\sum_{w'} \text{count}(w')^\alpha},$$

donde normalmente

$$\boxed{\alpha = 0{,}75}.$$

Esto aumenta ligeramente la probabilidad relativa de seleccionar palabras poco frecuentes respecto a utilizar directamente sus frecuencias en el corpus.

Función de pérdida de Skip-gram

Para un par positivo (w, c_{pos}) y k ejemplos negativos, queremos:

$$P(+ \mid w, c_{\text{pos}}) \rightarrow 1$$

y

$$P(- \mid w, c_{\text{neg},i}) \rightarrow 1.$$

Función de pérdida de Skip-gram

Para un par positivo (w, c_{pos}) y k ejemplos negativos, queremos:

$$P(+ \mid w, c_{\text{pos}}) \rightarrow 1$$

y

$$P(- \mid w, c_{\text{neg},i}) \rightarrow 1.$$

Esto conduce a la función de pérdida:

$$\boxed{\mathcal{L} = -\left[\log \sigma(c_{\text{pos}}^T w) + \sum_{i=1}^{k} \log \sigma(-c_{\text{neg},i}^T w)\right].}$$

Función de pérdida de Skip-gram

Para un par positivo (w, c_{pos}) y k ejemplos negativos, queremos:

$$P(+ \mid w, c_{\text{pos}}) \to 1$$

y

$$P(- \mid w, c_{\text{neg},i}) \to 1.$$

Esto conduce a la función de pérdida:

$$\boxed{\mathcal{L} = -\left[\log \sigma(c_{\text{pos}}^T w) + \sum_{i=1}^{k} \log \sigma(-c_{\text{neg},i}^T w)\right].}$$

Minimizar $\mathcal{L}$ equivale a:

aumentar la similitud con contextos reales

y

disminuir la similitud con contextos negativos.

Entrenamiento de Word2Vec

La función de pérdida se minimiza utilizando **descenso por gradiente estocástico (SGD)**.

Para cada ejemplo de entrenamiento:

$$(w, c_{\text{pos}}, c_{\text{neg},1}, \ldots, c_{\text{neg},k})$$

se actualizan los embeddings correspondientes:

$$\boxed{\begin{array}{c} w \\ c_{\text{pos}} \\ c_{\text{neg},1}, \ldots, c_{\text{neg},k} \end{array}}$$

Entrenamiento de Word2Vec

La función de pérdida se minimiza utilizando **descenso por gradiente estocástico (SGD)**.

Para cada ejemplo de entrenamiento:

$$(w, c_{\text{pos}}, c_{\text{neg},1}, \ldots, c_{\text{neg},k})$$

se actualizan los embeddings correspondientes:

$$\boxed{\begin{array}{c} w \\ c_{\text{pos}} \\ c_{\text{neg},1}, \ldots, c_{\text{neg},k} \end{array}}$$

Después de procesar muchos contextos del corpus:

$$\boxed{\text{palabras con contextos similares} \longrightarrow \text{vectores similares}}$$

La tarea de clasificación es solo el mecanismo utilizado para **aprender los embeddings**.

Word2Vec aprende embeddings estáticos

Una característica importante de Word2Vec es que aprende **un único vector por palabra**.

$$\boxed{w \longrightarrow w}$$

Por ejemplo,

$$\text{bank} \longrightarrow w_{\text{bank}}.$$

Word2Vec aprende embeddings estáticos

Una característica importante de Word2Vec es que aprende **un único vector por palabra**.

$$\boxed{w \longrightarrow w}$$

Por ejemplo,

$$\text{bank} \longrightarrow w_{\text{bank}}.$$

El vector es el mismo independientemente del contexto:

"deposit money in the bank"

"sit on the river bank".

Word2Vec aprende embeddings estáticos

Una característica importante de Word2Vec es que aprende **un único vector por palabra**.

$$\boxed{w \longrightarrow w}$$

Por ejemplo,

$$\text{bank} \longrightarrow w_{\text{bank}}.$$

El vector es el mismo independientemente del contexto:

"deposit money in the bank"

"sit on the river bank".

Limitación

Word2Vec no genera una representación diferente para cada significado o contexto de una palabra.

Modelos posteriores utilizan **embeddings contextuales**, donde la representación depende de la oración en la que aparece la palabra.

Otros embeddings estáticos

Word2Vec no es el único método para obtener embeddings estáticos.

FastText
Representa una palabra utilizando también sus *n*-gramas de caracteres.
Por ejemplo, con $n = 3$:

$$\text{where} \longrightarrow \{<\texttt{wh}, \texttt{whe}, \texttt{her}, \texttt{ere}, \texttt{re}>\}.$$

Esto permite representar mejor palabras raras o incluso **palabras no observadas durante el entrenamiento**.

Otros embeddings estáticos

Word2Vec no es el único método para obtener embeddings estáticos.

FastText
Representa una palabra utilizando también sus n-gramas de caracteres.
Por ejemplo, con $n = 3$:

$$\text{where} \longrightarrow \{<\texttt{wh}, \texttt{whe}, \texttt{her}, \texttt{ere}, \texttt{re}>\}.$$

Esto permite representar mejor palabras raras o incluso **palabras no observadas durante el entrenamiento**.

GloVe
Aprende embeddings utilizando estadísticas globales de **coocurrencia de palabras** en el corpus.

Word2Vec, FastText y GloVe producen **embeddings estáticos**.

Word2Vec: resumen

Corpus → pares palabra–contexto → clasificación → embeddings

1. Elegimos una palabra objetivo w.
2. Las palabras cercanas generan ejemplos positivos.
3. Muestreamos palabras aleatorias como ejemplos negativos.
4. Comparamos los embeddings mediante el producto punto.
5. Ajustamos los vectores para distinguir ambos casos.

Word2Vec: resumen

Corpus → pares palabra–contexto → clasificación → embeddings

1. Elegimos una palabra objetivo w.
2. Las palabras cercanas generan ejemplos positivos.
3. Muestreamos palabras aleatorias como ejemplos negativos.
4. Comparamos los embeddings mediante el producto punto.
5. Ajustamos los vectores para distinguir ambos casos.

El resultado es una representación

$$w \longrightarrow \mathbf{w} \in \mathbb{R}^d$$

densa, de baja dimensión y capaz de capturar
relaciones semánticas entre palabras.

Resumen

Para aplicar aprendizaje de máquina a textos necesitamos

convertir texto en números

Dos grandes familias:

$$\text{Texto} \rightarrow \text{BoW} \rightarrow \text{TF-IDF}$$

representaciones dispersas

$$\text{Texto} \rightarrow \text{contextos} \rightarrow \text{embeddings}$$

representaciones densas

En ambos casos obtenemos vectores que podremos utilizar para

comparar, agrupar y clasificar textos.

Siguiente paso: agrupamiento de textos

Ahora podemos representar cada documento como un vector:

$$d_i \longrightarrow x_i.$$

Por tanto, los métodos de aprendizaje no supervisado que ya conocemos pueden aplicarse directamente:

$$\{x_1, \ldots, x_n\} \xrightarrow{K\text{-means}} \text{grupos de documentos}$$

Tema 4.3: **Agrupamiento de textos**

¡Gracias!